DBMS Practical Queries Repository

Repository Name Suggestions

- dbms-practicals
- dbms-lab-queries
- mysql-dbms-practicals
- engineering-dbms-practical
- dbms-practical-exam-prep

Recommended Repository Structure

```
DBMS-Practicals/
│
├── README.md
├── Assignment-1-Database-Creation/
│   └── assignment1.sql
├── Assignment-2-Views-Indexing/
│   └── assignment2.sql
├── Assignment-3-SQL-Queries/
│   └── assignment3.sql
├── Assignment-4-Joins/
│   └── assignment4.sql
├── Assignment-5-Procedures-Functions/
│   └── assignment5.sql
├── Assignment-6-Triggers-Cursors/
│   └── assignment6.sql
├── Assignment-7-MongoDB/
│   └── assignment7.js
├── Assignment-8-Case-Study/
│   └── assignment8.sql
├── Assignment-9-Backup-Recovery/
│   └── assignment9.sql
└── schema.sql
```

README.md

```
# DBMS Practical Queries

This repository contains DBMS practical programs and SQL queries for engineering
practical examinations.

## Topics Covered
- Database Creation
- Constraints
- Views
- Indexing
- SQL Queries
- Joins
- Subqueries
- Procedures
- Functions
- Triggers
- MongoDB Basics
- Backup & Recovery

## Tools Used
- MySQL Workbench 8.0
- MongoDB

## How to Run
1. Open MySQL Workbench
2. Run schema.sql
3. Open assignment files
4. Execute queries using Ctrl + Enter

## Author
Niraj
```

---

schema.sql

```
CREATE DATABASE demo1;
USE demo1;

CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(50),
```

```
  Email VARCHAR(50),
  Age INT,
  Address VARCHAR(50)
);

CREATE TABLE Instructor (
  InstructorID INT PRIMARY KEY,
  Name VARCHAR(50),
  Email VARCHAR(50),
  Department VARCHAR(50)
);

CREATE TABLE Course (
  CourseID INT PRIMARY KEY,
  CourseName VARCHAR(50),
  InstructorID INT,
  Credits INT,
  FOREIGN KEY (InstructorID)
  REFERENCES Instructor(InstructorID)
);

CREATE TABLE Enrollment (
  EnrollmentID INT PRIMARY KEY,
  StudentID INT,
  CourseID INT,
  EnrollmentDate DATE,
  FOREIGN KEY (StudentID)
  REFERENCES Student(StudentID),
  FOREIGN KEY (CourseID)
  REFERENCES Course(CourseID)
);
```

Assignment 1 - Database Creation & Constraints

```
-- Create Database
CREATE DATABASE demo1;
USE demo1;

-- Create Student Table
CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(50),
  Email VARCHAR(50) UNIQUE,
  Age INT,
```

```
  Address VARCHAR(50)
);

-- Create Instructor Table
CREATE TABLE Instructor (
  InstructorID INT PRIMARY KEY,
  Name VARCHAR(50),
  Email VARCHAR(50),
  Department VARCHAR(50)
);

-- Create Course Table
CREATE TABLE Course (
  CourseID INT PRIMARY KEY,
  CourseName VARCHAR(50),
  InstructorID INT,
  Credits INT,
  FOREIGN KEY (InstructorID)
  REFERENCES Instructor(InstructorID)
);

-- Create Enrollment Table
CREATE TABLE Enrollment (
  EnrollmentID INT PRIMARY KEY,
  StudentID INT,
  CourseID INT,
  EnrollmentDate DATE,
  FOREIGN KEY (StudentID)
  REFERENCES Student(StudentID),
  FOREIGN KEY (CourseID)
  REFERENCES Course(CourseID)
);

-- Display Table Structure
DESC Student;
DESC Instructor;
DESC Course;
DESC Enrollment;

-- Show Tables
SHOW TABLES;
```

Assignment 2 - Views & Indexing

```sql
-- Create Simple View
CREATE VIEW StudentView AS
SELECT Name, Email
FROM Student;

-- Display View
SELECT * FROM StudentView;

-- Update View
UPDATE StudentView
SET Name = 'Rohan'
WHERE Name = 'Rahul';

-- Delete Through View
DELETE FROM StudentView
WHERE Name = 'Sneha';

-- Create Index
CREATE INDEX idx_email
ON Student(Email);

-- Show Indexes
SHOW INDEX FROM Student;

-- Drop View
DROP VIEW StudentView;
```

Assignment 3 - SQL Queries

```sql
-- Display All Students
SELECT * FROM Student;

-- Specific Columns
SELECT Name, Age
FROM Student;

-- WHERE Clause
SELECT *
FROM Student
WHERE Age > 20;
```

```sql
-- ORDER BY ASC
SELECT *
FROM Student
ORDER BY Age ASC;

-- ORDER BY DESC
SELECT *
FROM Student
ORDER BY Age DESC;

-- DISTINCT
SELECT DISTINCT Address
FROM Student;

-- LIKE Operator
SELECT *
FROM Student
WHERE Name LIKE 'R%';

-- BETWEEN Operator
SELECT *
FROM Student
WHERE Age BETWEEN 18 AND 22;

-- COUNT Function
SELECT COUNT(*) AS TotalStudents
FROM Student;

-- AVG Function
SELECT AVG(Age)
FROM Student;

-- MAX Function
SELECT MAX(Age)
FROM Student;

-- GROUP BY
SELECT CourseID, COUNT(*) AS TotalStudents
FROM Enrollment
GROUP BY CourseID;

-- HAVING Clause
SELECT CourseID, COUNT(*) AS TotalStudents
FROM Enrollment
GROUP BY CourseID
HAVING COUNT(*) > 1;
```

```sql
-- Subquery
SELECT Name
FROM Student
WHERE Age > (
  SELECT AVG(Age)
  FROM Student
);
```

Assignment 4 - Joins

```sql
-- INNER JOIN
SELECT s.Name, c.CourseName
FROM Student s
INNER JOIN Enrollment e
ON s.StudentID = e.StudentID
INNER JOIN Course c
ON e.CourseID = c.CourseID;

-- LEFT JOIN
SELECT s.Name, e.CourseID
FROM Student s
LEFT JOIN Enrollment e
ON s.StudentID = e.StudentID;

-- RIGHT JOIN
SELECT s.Name, e.CourseID
FROM Student s
RIGHT JOIN Enrollment e
ON s.StudentID = e.StudentID;

-- FULL OUTER JOIN USING UNION
SELECT s.Name, e.CourseID
FROM Student s
LEFT JOIN Enrollment e
ON s.StudentID = e.StudentID

UNION

SELECT s.Name, e.CourseID
FROM Student s
RIGHT JOIN Enrollment e
ON s.StudentID = e.StudentID;
```

Assignment 5 - Procedures & Functions

```
-- Procedure
DELIMITER //

CREATE PROCEDURE ShowStudents()
BEGIN
  SELECT * FROM Student;
END //

DELIMITER ;

-- Execute Procedure
CALL ShowStudents();

-- Function
DELIMITER //

CREATE FUNCTION StudentCount()
RETURNS INT
DETERMINISTIC
BEGIN
  DECLARE total INT;

  SELECT COUNT(*)
  INTO total
  FROM Student;

  RETURN total;
END //

DELIMITER ;

-- Execute Function
SELECT StudentCount();
```

Assignment 6 - Triggers & Cursors

```
-- Trigger
DELIMITER //

CREATE TRIGGER student_trigger
```

```
BEFORE INSERT ON Student
FOR EACH ROW
BEGIN
  SET NEW.Name = UPPER(NEW.Name);
END //

DELIMITER ;

-- Insert Data To Test Trigger
INSERT INTO Student VALUES
(10, 'niraj', 'n@gmail.com', 20, 'Pune');

-- Check Output
SELECT * FROM Student;

-- Cursor Example
DELIMITER //

CREATE PROCEDURE CursorDemo()
BEGIN
  DECLARE done INT DEFAULT FALSE;
  DECLARE sname VARCHAR(50);

  DECLARE cur CURSOR FOR
  SELECT Name FROM Student;

  DECLARE CONTINUE HANDLER FOR NOT FOUND
  SET done = TRUE;

  OPEN cur;

  read_loop: LOOP
    FETCH cur INTO sname;

    IF done THEN
      LEAVE read_loop;
    END IF;

    SELECT sname;
  END LOOP;

  CLOSE cur;
END //

DELIMITER ;
```

Assignment 7 - MongoDB Queries

```
-- Create Database
use college

-- Insert One Document
db.students.insertOne({
  name: "Niraj",
  age: 20,
  city: "Pune"
})

-- Insert Many
db.students.insertMany([
  {name: "Rahul", age: 22},
  {name: "Sneha", age: 19}
])

-- Find Documents
db.students.find()

-- Find Specific
db.students.find({age: 20})

-- Update Document
db.students.updateOne(
  {name: "Niraj"},
  {$set: {city: "Mumbai"}}
)

-- Delete Document
db.students.deleteOne({name: "Rahul"})
```

Assignment 8 - Case Study Queries

```
-- Count Students Per Course
SELECT CourseID, COUNT(*)
FROM Enrollment
GROUP BY CourseID;

-- Department Wise Instructor Count
SELECT Department, COUNT(*)
```

```sql
FROM Instructor
GROUP BY Department;

-- Student and Course Details
SELECT s.Name, c.CourseName
FROM Student s
JOIN Enrollment e
ON s.StudentID = e.StudentID
JOIN Course c
ON e.CourseID = c.CourseID;
```

Assignment 9 - Backup & Recovery

```sql
-- Backup Database
mysqldump -u root -p demo1 > backup.sql

-- Restore Database
mysql -u root -p demo1 < backup.sql

-- Export Table
SELECT *
INTO OUTFILE 'students.csv'
FIELDS TERMINATED BY ','
FROM Student;
```

GitHub Commands

```
git init
git add .
git commit -m "Added DBMS practical queries"
git branch -M main
git remote add origin YOUR_GITHUB_REPO_LINK
git push -u origin main
```

```
git init
git add .
git commit -m "Added DBMS practical queries"
git branch -M main
```

```
git remote add origin YOUR_GITHUB_REPO_LINK
git push -u origin main
```

Extra Tips

- Keep one query per section
- Add comments before queries
- Use proper formatting
- Push regularly to GitHub
- Add screenshots later if needed